RL Report FAQ

CS7641: Machine Learning

Minimum validation requirements

- **Hyperparameters:** For *each* model you submit (VI, PI, SARSA, Q-Learning, and any DQN/Rainbow EC), you must validate **at least two (2)** hyperparameters. More is encouraged when sensible. Examples that count: learning-rate schedule (α_0, decay law), exploration schedule (ε_0, floor, decay horizon), discount γ, discretization resolution (per-feature bins/clamps), initialization strategy (optimistic vs. zero), target update period (for DQN), replay buffer size, batch size. Report *ranges*, *sampling distributions*, and the *final selection* with justification.
- **Seeds:** Use **around 5 independent random seeds** for *every* model/config you compare. Typically in production, around 30-50 are more appropriate to show robustnes, but 5 is fine for this report. Aggregate with mean and a variability bands (e.g., IQR or 95% CI). Report the exact seed list and the total wall-clock.

What's the last assignment about?

High-level. You will perform experiments on **two MDPs** and analyze their properties:

- **Model-based**: Value Iteration (VI) and Policy Iteration (PI).
- **Model-free**: **SARSA** and **Q-Learning** (any standard tabular variant).

For Spring 2026, the two problems are provided in the assignment description (e.g., `Blackjack-v1` and `CartPole-v1`). CartPole requires discretization for VI/PI and for tabular methods.

What libraries should I use?

Use `bettermdptools` (Python) for MDP utilities and your plotting library of choice. Gymnasium environments are common and acceptable, but not mandatory. Integrate your environment with your tooling in a reproducible way (clear seeds, run scripts, figure generation commands).

Where does Q-Learning fit?

Additive to SARSA. Implement **tabular Q-Learning** and compare directly against **tabular SARSA** on both MDPs. Keep the state-action value update definitions explicit in your report:

$$\text{SARSA (on-policy):} \quad Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_{t+1} + \gamma\, Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\Big]$$

$$\text{Q-Learning (off-policy):} \quad Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big]$$

Report *sample efficiency*, *stability*, and *final return*, and discuss how on- vs. off-policy targets change learning dynamics. If you use *Double Q-Learning* (tabular) to reduce overestimation, state it clearly.

What are these "interesting properties"?

Before running, examine the MDP structure (state space, transitions, rewards). Then hypothesize:

- How stochasticity or sparsity might impact exploration and convergence.
- Whether discretization coarseness induces suboptimal policies or instability.
- Sensitivity to learning rate (α), discount (γ), and exploration schedules.

Your goal is to use the problems as *probes* to reveal algorithmic behavior you can analyze and explain.

CartPole discretization: concrete guidance

CartPole is continuous; tabular methods and DP require a discrete state space. Use *non-uniform* binning that prioritizes the pole angle and angular velocity:

- **State features & suggested clamps**:
  - Cart position $x \in [-2.4, 2.4]$ (3 to 5 bins).
  - Cart velocity $\dot{x}$ clamp to $[-3, 3]$ (3 to 7 bins).
  - Pole angle $\theta \in [-0.2, 0.2]$ rad (6 to 12 bins).
  - Pole angular velocity $\dot{\theta}$ clamp to $[-3.5, 3.5]$ (6 to 12 bins).
- **Starter grids**: $(1, 1, 6, 12)$ or $(3, 3, 6, 12)$. Increase angle-/angular-velocity resolution first.
- **Ablate the grid**: run coarse to fine and document policy quality, stability, wall-clock, and convergence.

State your bin edges explicitly in the report to ensure reproducibility.

Convergence: criteria you can defend

Depending on method, choose criteria that match the objects being optimized:

- **VI/PI**: $\max_s |V_{k+1}(s) - V_k(s)| < \delta$ for m consecutive sweeps; track sweep counts and wall-clock.
- **Tabular RL**: running mean episodic return plateaus; or $\max_{s,a} |Q_{t+1}(s, a) - Q_t(s, a)| < \delta$ for m consecutive episodes.

Report *both* an algorithmic convergence indicator and a task metric (e.g., average episode length in CartPole).

What artifacts should I include?

Support claims with data:

- **Learning curves**: return vs. episodes (SARSA, Q-Learning).
- **Convergence diagnostics**: ΔV or ΔQ vs. iteration/episode.
- **Policies/Value maps**: heatmaps or matrices (Blackjack is amenable).
- **Discretization study**: grid size vs. performance and compute.

Continuous environments: workflow

For VI/PI you must discretize. A pragmatic workflow:

1. Use model-free (SARSA/Q-Learning) first to *explore* and get intuition for state scales and useful binning.
2. Log transitions and rewards to estimate model dynamics if you wish to sanity-check your DP setup.
3. Consider *tile coding* or non-uniform bins as resolution increases.

Model-free specifics: exploration & evaluation

Discuss ε-greedy or Boltzmann exploration and justify your schedule. Keep ε nonzero longer than you expect; brittle discretizations need sustained exploration. Compare SARSA vs. Q-Learning under the *same* exploration schedule to isolate on-/off-policy effects.

Variance across runs

Average across **5 independent seeds** for each model/config. Align curves by episodes (or pad to the right) and show mean $\pm$ variability (e.g., shaded IQR or 95% CI). Report seeds and all randomization points (env, replay orders if used, tie-breaks). Do not submit single-seed results.

Q-Learning: practical defaults that work

For tabular baselines (Blackjack, discretized CartPole):

- $\gamma \in \{0.95, 0.99\}$; start with 0.99 on CartPole.
- α: start 0.5 and decay (e.g., linear to 0.1) or use visit-based $\alpha = 1/(1 + \text{visits}(s, a))$.
- ε-greedy: $1.0 \to 0.01$ over 5 to 20k steps; keep a floor (e.g., 0.01).
- Consider **Double Q-Learning** (two tables, alternating targets) to reduce overestimation.

State-action initializations matter in sparse feedback tasks; try optimistic Q_0 for faster exploration, but report it.

Stop doing exhaustive grid search. Do this instead.

You do not have the budget to sweep every combination; you also shouldn't guess. Use a staged, *device-aware* strategy that converges quickly on decent settings and then refines.

Stage 1: Coarse random search with early stopping

- Sample N candidates over *log-scaled* ranges (e.g., $\alpha \sim 10^{U(-3,0)}$, ε-floor $\in [0.005, 0.05]$, decay horizons, $\gamma \in [0.95, 0.999]$).
- **Early stop**: run only E_{pilot} episodes (small), discard bottom half by interim return. This is Successive Halving in spirit.

Stage 2: Successive Halving / Hyperband-style promotion

- Allocate more episodes to the top k from Stage 1; prune aggressively.
- Promotion continues until a small champion set remains (compute-budget matched to your *actual* device/VM).

Stage 3: Local refinement around winners

- Narrow ranges around best settings (e.g., $\alpha \pm 2\times$; decay horizon $\pm 25\%$).
- Try **one-at-a-time sensitivity** to understand which knobs move the metric vs. noise.

Stage 4 (optional): Population-based training (PBT) light

- Maintain a small population; periodically copy weights/hyperparams from the top performer to a weaker one and *mutate* hyperparams slightly (e.g., ± 10–20% on α, ε-decay length).
- This is cheap tabularly and adapts schedules online.

Heuristics that actually help

- Tune **exploration schedules** *first*; learning rate matters less if you never visit useful states.
- Couple α decay to visit counts or episode index; too-fast decay yields stagnation, too-slow yields noise.
- If curves are bouncy but trend upward, try Double Q-Learning before retuning everything.
- Always re-check discretization; many "hyperparameter" issues are actually state aliasing from coarse bins.

How to write about hyperparameters

- Describe the *search protocol* (stages, budgets, pruning rule), not just the final numbers.
- Provide ranges, sampling distributions, and episode budgets per stage.
- Include the *time cost*: total episodes, wall-clock on your device; justify trade-offs.
- Explicitly list the **at least 2** validated hyperparameters per model and summarize sensitivity (what mattered vs. noise).

Extra Credit (Rainbow DQN ablation)

For up to 5 points, implement and analyze one Rainbow component on CartPole (e.g., Double DQN, Dueling, Prioritized Replay, Noisy Nets, n-step, or C51). Compare against a *vanilla* DQN baseline and your tabular methods. Explicitly state:

- Which ablation you implemented and how it changes targets/architecture.
- Stabilization choices (target network period, replay size, batch size, optimizer).
- Where it outperforms or fails vs. tabular methods and why.
- Aggregate EC results over **5 seeds**; do not submit single-seed EC.

If you attempt EC, keep it *separate* and time-boxed so the core (VI/PI/SARSA/Q-Learning) is complete.

Common failure modes (read this before filing bugs)

- ε decays to near-zero too early $\Rightarrow$ premature exploitation of a bad policy.
- Discretization too coarse $\Rightarrow$ conflates recoverable and unrecoverable states; learning appears "noisy".
- Unclamped velocities $\Rightarrow$ huge state space with sparse revisits; nothing converges.
- Over-tuning α while ignoring exploration or bins.

Submission and policy reminders

Follow the PDF, Overleaf, provenance, and AI-use requirements in the Spring 2026 brief; include seeds, exact run scripts, and figure-generation code so results are reproducible.

One-page quick-start

- **CartPole bins**: $(x, \dot{x}, \theta, \dot{\theta}) = (3, 3, 8, 12)$ with clamps $[-2.4, 2.4], [-3, 3], [-0.2, 0.2], [-3.5, 3.5]$.
- **Q-Learning defaults**: $\gamma = 0.99$, $\alpha_0 = 0.5 \to 0.1$ (linear decay), $\varepsilon : 1.0 \to 0.01$ over 10k steps, floor 0.01; Double Q optional.
- **Search protocol**: Stage 1 random $N = 24$ (pilot 200 episodes) $\to$ keep top 8; Stage 2 add 400 episodes $\to$ keep top 3; Stage 3 local refine $\pm 2\times$ on α, $\pm 25\%$ on decay horizon; **report champion over 5 seeds**.

Remember: Use the MDPs to **explain** algorithm behavior, not just to hit a score. Your writeup should make clear *why* SARSA vs. Q-Learning differs under your discretization, exploration, and tuning choices.